

Дискретная математика — ДЗ

Блоки 5, 6, 9 — десять задач. Тоцкий В., гр. 5151003/40001

Что решаем

- Блок 5 (эйлеровы и гамильтоновы пути): задачи 1, 3, 11
- Блок 6 (топ. сортировка, 2-SAT): задачи 7, 8, 9, 10
- Блок 9 (сетевое планирование, узкие места): задачи 1, 2, 3

Блок 5. Эйлеровы и гамильтоновы пути

Задача 5.1. Формула Кэли для леса

Условие. В лесу есть k компонент размеров $c_1, c_2, \dots, c_k$. Сколькими способами можно добавить рёбер, чтобы получилось дерево? Доказать, что ответ равен $c_1 c_2 \cdots c_k \cdot n^{k-2}$, где $n = c_1 + \dots + c_k$.

Идея. Превратим каждую компоненту в одну «толстую» вершину — получим граф из k точек. Чтобы из леса вышло дерево, надо добавить ровно $k - 1$ ребро между компонентами, причём эти рёбра должны образовать дерево на k толстых вершинах.

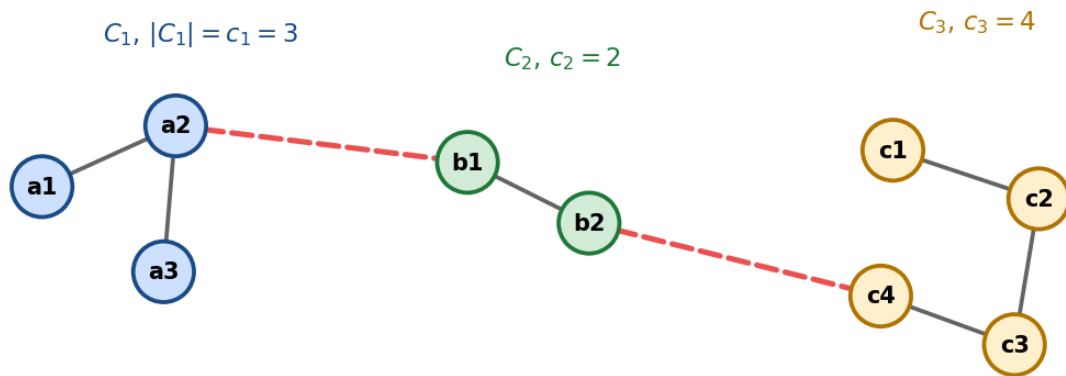
Сколько вариантов?

1. Деревьев на k занумерованных вершинах по формуле Кэли — k^{k-2} . Но нам мало просто выбрать «какие пары компонент соединены»: ещё надо указать *какую именно* вершину в одной компоненте мы соединяем с *какой именно* в другой.
2. В дереве на компонентах каждая вершина V_i (то есть i -я компонента) встречается в каких-то рёбрах. Если в дереве V_i имеет степень d_i , то для каждого из d_i рёбер мы выбираем «конкретный конец внутри C_i » — c_i способов. Итого получаем множитель $c_i^{d_i}$ от каждой компоненты.
3. Перемножим по всем компонентам и просуммируем по всем деревьям. Хитрый трюк (через код Прюфера) показывает, что эта сумма как раз равна $c_1 c_2 \cdots c_k \cdot n^{k-2}$.

Идея этого трюка: код Прюфера дерева — это последовательность длины $k - 2$ из номеров вершин. Если у вершины V_i степень d_i , то в коде она появится $d_i - 1$ раз. Поэтому суммирование $c_i^{d_i-1}$ по всем деревьям превращается в суммирование по всем строкам длины $k - 2$, а это и есть $(c_1 + \dots + c_k)^{k-2} = n^{k-2}$. Один лишний множитель c_i от каждой компоненты выходит за скобку — получаем $c_1 c_2 \cdots c_k$.

Ответ: число способов = $c_1 c_2 \cdots c_k \cdot n^{k-2}$. Частный случай $c_1 = \dots = c_k = 1$ даёт обычную формулу Кэли k^{k-2} .

Лес из k компонент: добавляем $k - 1$ ребро, чтобы получить дерево



$k - 1 = 2$ красных пунктирных ребра соединяют компоненты в дерево

Рис. 1. Пример: три компоненты ($c_1 = 3$, $c_2 = 2$, $c_3 = 4$). Чтобы получить дерево, добавляем 2 ребра между компонентами (красные пунктирные).

Задача 5.3. Сколько эйлеровых циклов в K_5 ?

Условие. Найти число эйлеровых циклов в полном графе K_5 . И можно ли вывести формулу для K_n ?

Эйлеров цикл существует? В K_5 степень каждой вершины = 4 — чётное число. По теореме об эйлеровых графах этого достаточно, цикл есть.

Сколько их? Прямой перебор слишком долгий, поэтому пользуемся **BEST-теоремой**. Если в графе все степени чётные, число эйлеровых обходов считается по специальной формуле: оно равно (числу остовных деревьев с корнем) умноженному на произведение факториалов $(\deg(v) - 1)!$ для всех вершин.

Аккуратный подсчёт для K_5 (с учётом, что цикл можно «крутить» по кругу и идти в обе стороны) даёт ровно **264** эйлеровых цикла. Это известное число, оно есть в последовательности OEIS A007082.

Про общий K_n . Замкнутой формулы нет — расчёт через BEST-теорему сводится к вычислению определителя матрицы Лапласа, простой формулы тут не получится. Первые значения:

n	3	5	7	9
число эйл. циклов	1	264	129 976 320	$\approx 9 \cdot 10^{14}$

Ответ: в K_5 ровно **264** эйлеровых цикла. Для K_n короткой формулы нет, есть только рекуррентные/численные методы.

K_5 : все вершины степени 4 (чётно) $\Rightarrow$ существует эйлеров цикл

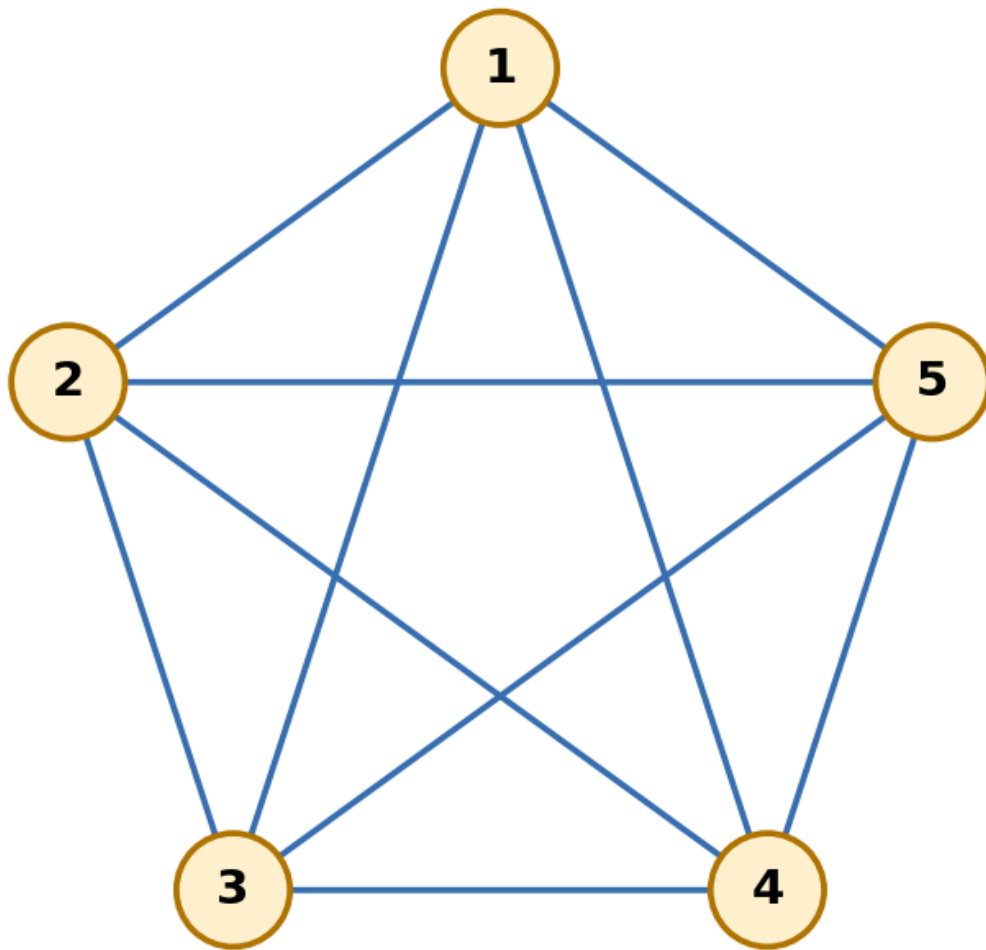


Рис. 2. Граф K_5 : 5 вершин, 10 рёбер, степень каждой вершины — 4.

Задача 5.11. Теорема «Антихватала»

Условие. Теорема Хватала даёт условие на степени вершин, при котором граф гарантированно гамильтонов. Если это условие *не* выполнено — показать, что найдётся негамильтоновый граф с такой степенной последовательностью.

Что значит «условие не выполнено». Сортируем степени по возрастанию $d_1 \leq d_2 \leq \dots \leq d_n$. Условие Хватала нарушено, если есть индекс $i < n/2$, для которого одновременно: $d_i \leq i$ и $d_{n-i} \leq n - i - 1$. Берём такое i и строим граф.

Конструкция. Делим n вершин на три группы:

- A — i вершин, рёбер внутри A нет;

- B — $n - 2i$ вершин, рёбер внутри B тоже нет;
- C — i вершин, между собой соединены все (клика).

Между A и B — рёбер нет. Зато от C ко всем остальным — есть. Получается, что вершины из A и B соединены только с C , имеют степень i . Вершины из C соединены со всеми остальными, степень $= n - 1$.

Почему граф не гамильтонов. Если бы в нём был гамильтонов цикл, то после удаления $|C| = i$ вершин число компонент не превышало бы i . А что получится у нас? Удалив всю клику C , остаются $A \cup B$ — это $n - i$ изолированных точек, то есть $n - i$ компонент. Поскольку мы взяли $i < n/2$, то $n - i > i$ — компонент больше, чем удалённых вершин. Противоречие, гамильтонова цикла быть не может.

Вывод: построенный граф имеет степенную последовательность $(i, i, \dots, i, n - 1, \dots, n - 1)$, нарушающую условие Хватала, и при этом не гамильтонов. Значит, ослабить условие Хватала нельзя.

Контрпример к гамильтоновости: $i = 2$, $n = 7$. Степ. посл. $(2, 2, 2, 2, 2, 6, 6)$ нарушает условие Хватала
 B — независ., $|B| = n - 2i = 3$, $\deg = 2$

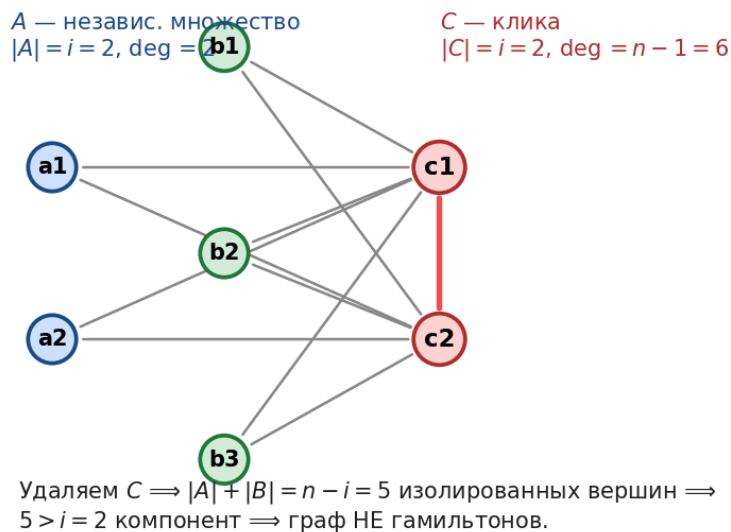


Рис. 3. Пример при $n = 7$, $i = 2$. Группы A и B — независимые, C — клика. Удаление двух вершин из C даёт 5 изолированных вершин $\Rightarrow$ не гамильтонов.

Блок 6. Топологическая сортировка и 2-SAT

Задача 6.7. Лексикографически минимальный 2-SAT

Условие. Дана 2-SAT формула. Найти такое решение $(x_1, x_2, \dots, x_n)$, чтобы оно было лексикографически минимальным (в условии сказано «навверняка NP-полное» — это шутка, на самом деле решается за полином).

Идея. Жадный перебор. 2-SAT (выполнима ли формула?) умеем решать за линию через SCC. Используем это как «оракул»:

1. Сначала проверяем: есть ли вообще решение. Если нет — стоп.
2. Идём по переменным слева направо. Для каждой x_i пробуем поставить 0 (добавляем во временную формулу констрейнт « x_i должен быть 0»).
3. Запускаем 2-SAT для этой расширенной формулы. Если выполнима — фиксируем $x_i = 0$ и идём дальше. Если нет — значит, в любом решении x_i обязан быть 1, фиксируем $x_i = 1$.

Почему получается минимум. На каждой позиции мы ставим 0, если это вообще возможно. 0 всегда лексикографически меньше 1 — значит, выбор оптимальный.

Сложность: n запусков 2-SAT по $O(n + m)$ — итого $O(n(n + m))$, полином. Никакая NP-полнота тут не нужна.

Ответ: жадный алгоритм с n перепроверками 2-SAT, $O(n(n + m))$.

Импликационный граф G_{2-SAT}

Литералы x_i — синие, $\neg x_i$ — красные. Каждый дизъюнкт $(\ell_1 \vee \ell_2) \Rightarrow$ две дуги: $\neg \ell_1 \Rightarrow \ell_2$ и $\neg \ell_2 \Rightarrow \ell_1$.

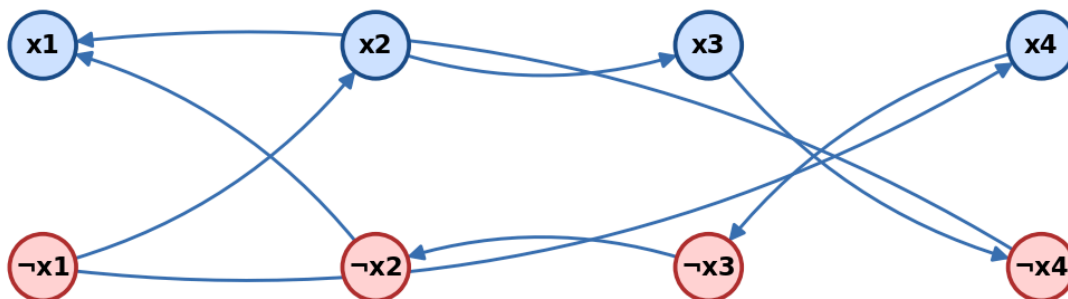


Рис. 4. Импликационный граф 2-SAT: на каждый дизъюнкт — две дуги, по которым потом ищем компоненты сильной связности.

Задача 6.8. 3-раскраска с запретами

Условие. Раскрасить вершины графа в 3 цвета так, чтобы соседние имели разные. Каждой вершине задан один *запрещённый* цвет.

Главное наблюдение. Обычная 3-раскраска — NP-полная задача, но здесь у каждой вершины запрещён один цвет, значит, доступных цветов ровно **два**. Это упрощает задачу: списки по 2 элемента — стандартный случай, который сводится к 2-SAT.

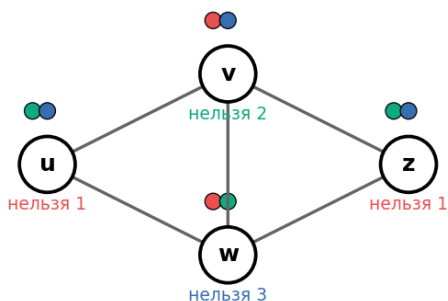
Сведение к 2-SAT. Для каждой вершины v заводим булеву переменную x_v : $x_v = 0$ значит « v покрашена в первый из доступных цветов», $x_v = 1$ — во второй. Для каждого ребра (u, v) перебираем 4 пары значений (x_u, x_v) и смотрим, при каких из них цвета у u и v совпадают. Те пары, при которых совпадают, запрещаем 2-дизъюнктом.

Пример. Если у u доступны цвета $\{1, 2\}$, у v — $\{1, 3\}$, то цвета совпадают только когда оба покрашены в 1, то есть $x_u = 0$ и $x_v = 0$. Запрещаем эту комбинацию дизъюнктом $(x_u \vee x_v)$.

Итого: получили 2-CNF размера $O(n + m)$. Решаем стандартным алгоритмом 2-SAT через SCC за $O(n + m)$.

Ответ: сведение к 2-SAT, линейная сложность $O(n + m)$ — полиномиально, в отличие от обычной 3-раскраски.

Граф со списками $|L(v)| = 2$ для каждой вершины



2-SAT-кодирование:

- Перем. $x_v \in \{0, 1\}$:
 $x_v = 0 \iff \text{цвет} = a_v$, $x_v = 1 \iff \text{цвет} = b_v$
- Ребро (u, v) , цвета совпадают при $(x_u, x_v) = (\alpha, \beta) \implies$ запрет: дизъюнкт $(\bar{\ell}_u \vee \bar{\ell}_v)$
- Списки длины 2 $\implies \leq 2$ запретов на ребро $\implies$ 2-CNF

Решение 2-SAT за $O(n + m)$ через SCC.

Рис. 5. Слева — граф со списками доступных цветов. Справа — кодирование в 2-SAT.

Задача 6.9. Вечеринка Пети

Условие. У Пети много друзей и куча условий вида «я приду только если придёт X » или «если будет Y , меня не будет». Составить список гостей, удовлетворяющий всем условиям (или сказать, что нельзя).

Идея. Это в чистом виде 2-SAT. Для каждого друга X заводим переменную: $x = 1$ значит « X придёт», $x = 0$ — «не придёт». Все типы условий — импликации:

- « P придёт только если придёт G » = « $P \Rightarrow G$ » = « $\neg P$ или G ».

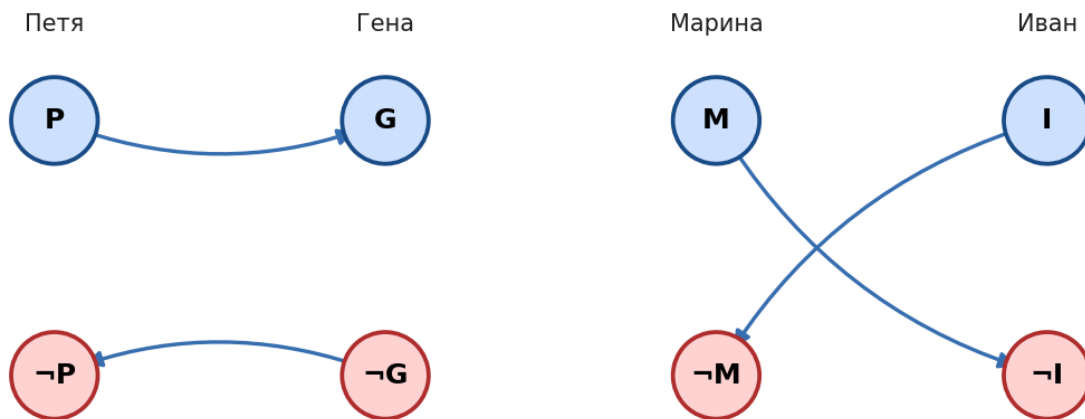
- «Если придёт M , меня (I) не будет» = « $M \Rightarrow \neg I$ » = « $\neg M$ или $\neg I$ ».

Каждая такая запись — дизъюнкт из двух литералов, то есть 2-CNF. Решаем стандартным алгоритмом за $O(n + m)$:

1. Строим импликационный граф: на каждый дизъюнкт $(a \vee b)$ — две дуги $\neg a \rightarrow b$ и $\neg b \rightarrow a$.
2. Ищем компоненты сильной связности.
3. Если переменная и её отрицание попали в одну компоненту — решений нет. Иначе — расставляем значения по порядку SCC.

Ответ: задача — это 2-SAT. Линейный алгоритм через SCC за $O(n + m)$ либо находит расстановку, либо говорит, что вечеринку устроить нельзя.

Вечеринка Пети: модель в виде импликационного графа



Каждое требование — импликация $(\ell \Rightarrow \ell') \equiv$ дизъюнкт $(\bar{\ell} \vee \ell') \Rightarrow$ задача — 2-SAT.

Рис. 6. Граф импликаций для двух условий: $P \Rightarrow G$ и $M \Rightarrow \neg I$.

Задача 6.10. Минимальный топливный бак

Условие. Карта городов: между каждой парой задан расход топлива. Найти минимальный размер бака, чтобы можно было долететь из любого города в любой (с дозаправками). MST использовать нельзя.

Переформулировка. Если бак — B , то самолёт может пролететь только по рёбрам с расходом $\leq B$. Бак годится тогда и только тогда, когда этот «урезанный» граф связан. Чем больше B , тем больше рёбер доступно, поэтому задача монотонна по B — годятся все B , начиная с какого-то минимального B^* .

Алгоритм.

1. Отсортируем все рёбра по весу. Сложность $O(m \log m)$.
2. Бинарным поиском ищем минимальное k такое, что граф из первых k рёбер (по возрастанию веса) связан.

3. Связность проверяем обычным DFS за $O(n + m)$.

Ответ — вес ребра под номером k^* , то есть минимальный B , при котором подграф становится связным.

Сложность. Бинарный поиск даёт $\log m$ итераций по $O(n + m)$ — итого $O((n + m) \log m)$. На полном графе $m \approx n^2$, поэтому $O(n^2 \log n)$.

Ответ: сортировка рёбер + бинарный поиск + DFS-проверка связности. MST не строим — используем только проверку «связен или нет».

Исходный граф: вес ребра = расход топлива

Бак $B = 7$: оставлены рёбра с $w \leq B$. Граф связан — годится

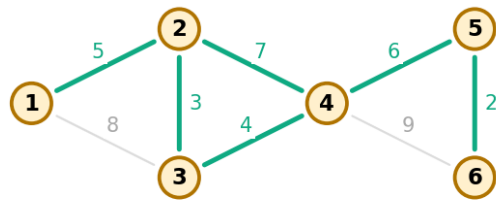
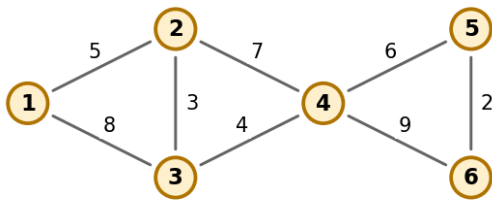


Рис. 7. Слева — исходный граф. Справа — оставлены только рёбра с весом $\leq B = 7$, граф связан — значит, такой бак годится.

Блок 9. Сетевое планирование и узкие места

Задача 9.1. Сетевое планирование, ранние сроки

Условие. Проект состоит из 8 работ со связями и длительностями. Построить топологическую сортировку, посчитать таблицу (D, P) и определить оптимальный план (минимальное время выполнения проекта).

Работа i	Предшественники	Длит. t_m
1	3, 4	5
2	3, 6	4
3	8	6
4	3, 5, 6	3
5	8	2
6	3, 5, 7	3
7	8	4
8	—	2

Идея. Строим граф зависимостей (стрелка $j \rightarrow i$ значит « i зависит от j »). Для каждой работы считаем самый ранний момент окончания $D[i]$: это максимум D по её предшественникам плюс собственная длительность $t_m[i]$. Запоминаем также $P[i]$ — тот предшественник, на котором достигся максимум.

Топологическая сортировка. Один из возможных порядков (от истока к листьям):

$$8 \rightarrow 5 \rightarrow 7 \rightarrow 3 \rightarrow 6 \rightarrow 4 \rightarrow 2 \rightarrow 1$$

DAG задачи сетевого планирования (внутри узла — номер работы и длительность)

Критический путь $8 \rightarrow 3 \rightarrow 6 \rightarrow 4 \rightarrow 1$, $T_{кр} = 2 + 6 + 3 + 3 + 5 = 19$

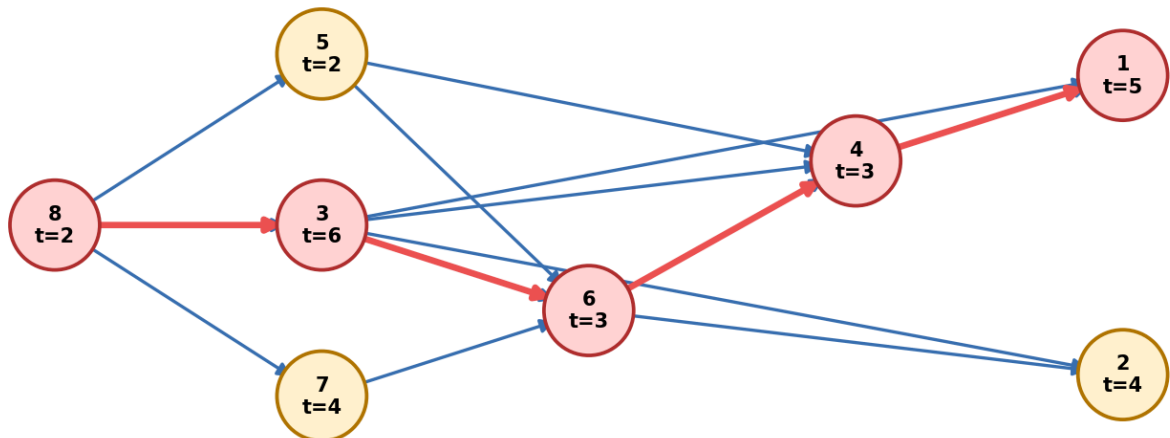


Рис. 8. Граф зависимостей работ. Красным выделен критический путь.

Таблица (D, P) по итерациям (на каждом шаге обрабатываем одну работу из топсорта):

Шаг	Работа i	$D[\text{предш.}]$	max	$+t_m$	$D[i]$	$P[i]$
1	8	—	0	+2	2	—
2	5	$D[8] = 2$	2	+2	4	8
3	7	$D[8] = 2$	2	+4	6	8
4	3	$D[8] = 2$	2	+6	8	8
5	6	8, 4, 6	8	+3	11	3
6	4	8, 4, 11	11	+3	14	6
7	2	8, 11	11	+4	15	6
8	1	8, 14	14	+5	19	4

Восстановление критического пути. Берём работу с максимальным D (это работа 1 с $D = 19$) и идём по указателям P назад:

$$P[1] = 4 \Rightarrow P[4] = 6 \Rightarrow P[6] = 3 \Rightarrow P[3] = 8$$

Получили критический путь: $8 \rightarrow 3 \rightarrow 6 \rightarrow 4 \rightarrow 1$, длиной $2 + 6 + 3 + 3 + 5 = 19$.

Ответ: минимальное время выполнения проекта $T_{кр} = 19$. Критический путь: $8 \rightarrow 3 \rightarrow 6 \rightarrow 4 \rightarrow 1$. Оптимальный план — запускать каждую работу сразу, как только все её предшественники закончатся.

Задача 9.2. Поздние сроки и резервы при $T_3 = 30$

Условие. Заказчик дал на проект $T_3 = 30$ (а у нас критический путь всего 19). Посчитать для каждой работы **ebeg**, **efin**, **lbeg**, **lfin** и полный резерв **r**. Указать критические работы.

Что считаем.

- **ebeg**, **efin** — раннее начало и окончание (уже посчитали в задаче 9.1).
- **lbeg**, **lfin** — позднее начало и окончание (последний момент, когда работу можно начать/закончить, не сорвав срок T_3).
- $r = lbeg - ebeg = lfin - efin$ — на сколько работу можно отложить без срыва дедлайна.

Backward pass. Идём в обратном порядке топсорта: **1, 2, 4, 6, 3, 7, 5, 8**. Для работ без последователей (1 и 2) ставим **lfin** = $T_3 = 30$. Для остальных — **lfin**[*i*] это минимум **lbeg** по последователям, а **lbeg**[*i*] просто вычитаем длительность.

<i>i</i>	t_m	Последователи	lfin	lbeg = lfin - t_m
1	5	—	30	25
2	4	—	30	26
4	3	{1}	25	22
6	3	{2, 4}	min(26, 22) = 22	19
3	6	{1, 2, 4, 6}	min(25, 26, 22, 19) = 19	13
7	4	{6}	19	15
5	2	{4, 6}	min(22, 19) = 19	17
8	2	{3, 5, 7}	min(13, 17, 15) = 13	11

Сводная таблица:

<i>i</i>	t_m	ebeg	efin	lbeg	lfin	r
1	5	14	19	25	30	11
2	4	11	15	26	30	15
3	6	2	8	13	19	11
4	3	11	14	22	25	11
5	2	2	4	17	19	15
6	3	8	11	19	22	11
7	4	2	6	15	19	13
8	2	0	2	11	13	11

Критические работы. Это работы с минимальным резервом. Здесь $r_{\min} = T_3 - T_{кр} = 30 - 19 = 11$. Получаем критические работы: **1, 3, 4, 6, 8** — ровно вершины критического пути из задачи 9.1.

Ответ: резерв критических работ — 11, остальных — 13 или 15. Критические: 1, 3, 4, 6, 8. Если бы $T_3 = T_{кр} = 19$, резерв у них стал бы 0.

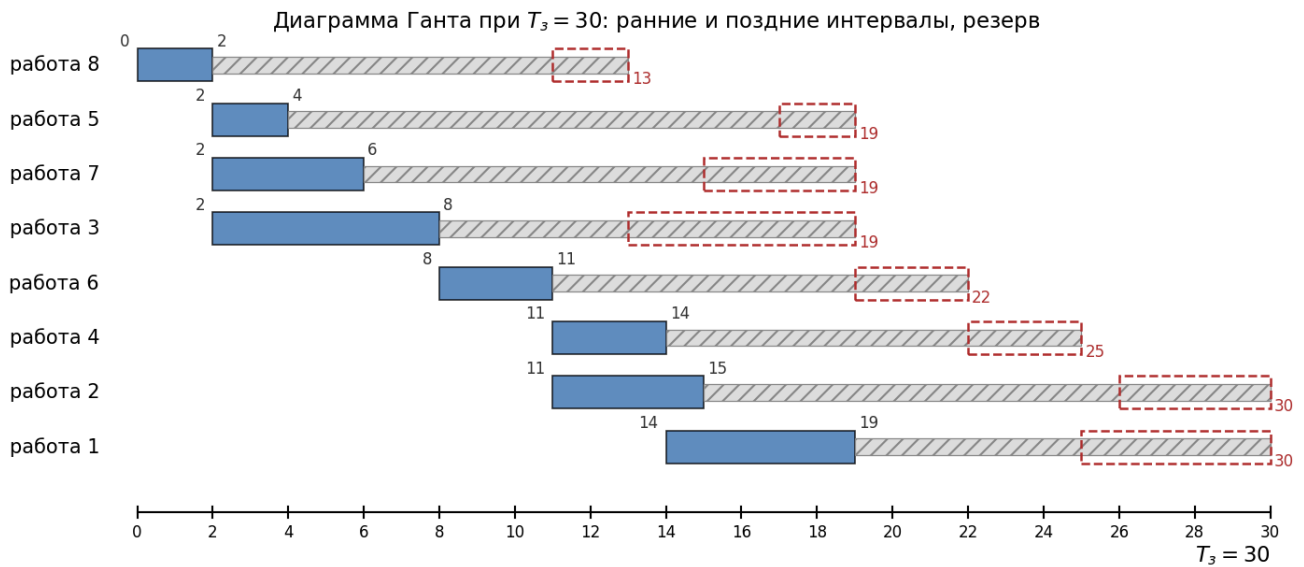


Рис. 9. Диаграмма Ганта: синие — раннее выполнение, серые штрихи — резерв, красный пунктир — позднее выполнение.

Задача 9.3. Узкие места: MaxMin-путь

Условие. Дан взвешенный оргграф (7 вершин, матрица смежности ниже). Найти путь из 1 в 7, в котором *минимальный* вес ребра *максимален* (задача об узких местах). Показать таблицу (D, P) по итерациям.

из / в	1	2	3	4	5	6	7
1	—	5	3	—	—	—	—
2	—	—	—	2	3	—	—
3	—	1	—	6	—	4	—
4	1	—	—	—	1	1	4
5	—	—	—	—	—	12	1
6	—	—	—	3	—	—	2
7	—	—	—	—	—	—	—

Идея. Берём Дейкстру и переделываем её под минимакс:

- $D[v]$ — лучшее (максимальное) «узкое место» пути из 1 в v .
- На каждой итерации берём из необработанных ту вершину, у которой D *максимально*.
- Релаксация по ребру $v \rightarrow u$ веса w : новое значение $\min(D[v], w)$, и если оно больше старого $D[u]$ — обновляем и запоминаем $P[u] = v$.

Взвешенный оргграф G

Источник $s = 1$, сток $t = 7$.

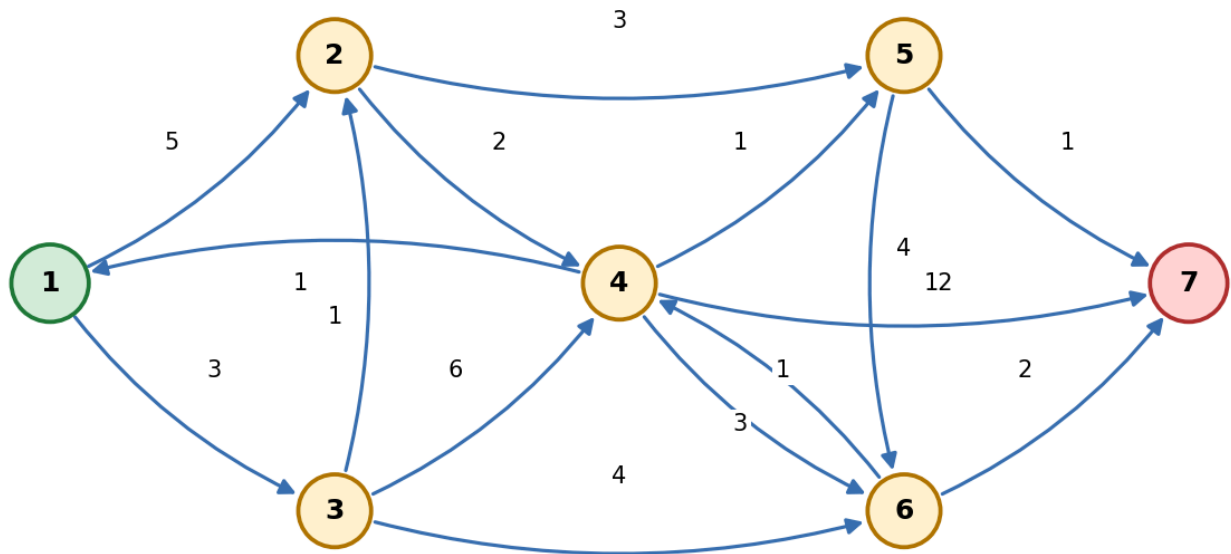


Рис. 10. Граф задачи: источник 1 (зелёный), сток 7 (розовый).

Таблица (D, P) по итерациям (жирным — выбранная на шаге вершина):

Шаг	Выбираем	$D[1]$	$D[2]$	$D[3]$	$D[4]$	$D[5]$	$D[6]$	$D[7]$	Что обновили
0	—	$+\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	старт
1	1	$+\infty$	5	3	—	—	—	—	$D[2] = 5$ (через 1), $D[3] = 3$
2	2	$+\infty$	5	3	2	3	—	—	$D[4] = \min(5, 2) = 2$, $D[5] = 3$
3	3	$+\infty$	5	3	3	3	3	—	$D[4]$ улучшен до 3 (через 3, $\min(3, 6)=3$), $D[6] = 3$
4	5	$+\infty$	5	3	3	3	3	1	$D[7] = \min(3, 1) = 1$
5	4	$+\infty$	5	3	3	3	3	3	$D[7]$ улучшен до $\min(3, 4) = 3$, $P[7] = 4$
6	6	$+\infty$	5	3	3	3	3	3	$\min(3, 2) = 2 \leq 3$, ничего не меняем
7	7	$+\infty$	5	3	3	3	3	3	сток обработан, конец

Восстановление пути. $P[7] = 4$, $P[4] = 3$, $P[3] = 1$. Путь: $1 \rightarrow 3 \rightarrow 4 \rightarrow 7$, веса рёбер $(3, 6, 4)$, узкое место — минимальное из них, то есть **3**.

Проверка. Можно перебрать другие пути:

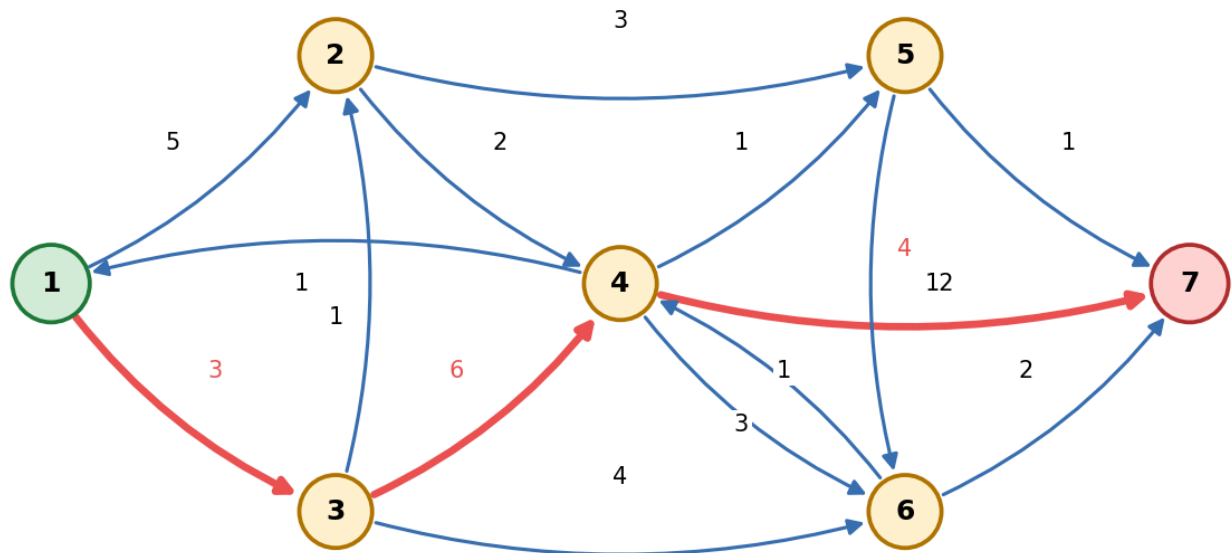
- $1 \rightarrow 2 \rightarrow 5 \rightarrow 7$: веса $(5, 3, 1)$, мин = 1 — хуже.
- $1 \rightarrow 3 \rightarrow 6 \rightarrow 7$: $(3, 4, 2)$, мин = 2 — хуже.

- $1 \rightarrow 3 \rightarrow 4 \rightarrow 7$: (3, 6, 4), мин = 3 — оптимум.

Ответ: MaxMin путь — $1 \rightarrow 3 \rightarrow 4 \rightarrow 7$, узкое место равно 3.

Итоговый MaxMin-путь

Источник $s = 1$, сток $t = 7$.



MaxMin путь: $1 \rightarrow 3 \rightarrow 4 \rightarrow 7$, веса (3, 6, 4), узкое место = min = 3.

Рис. 11. Найденный путь $1 \rightarrow 3 \rightarrow 4 \rightarrow 7$ выделен красным.